

Jawwid Chat — Project Brief for Implementation

Customer Success operating system inside the Jawwid app. Replaces WhatsApp for the CS team. Full design doc (v3) exists separately; this brief is the contract for what to build.

0. The one rule

One Family → One Primary Owner. The Owner is not a bottleneck. The system — not the employee — owns the family's history. People can change; the customer experience must not reset.

1. Operating model (decided — do not re-litigate)

Permanent Ownership + Configurable Coverage.

- Every family has exactly one fixed **Primary Owner** (one of 4 admins: أسماء، رحاب، زینب، دینا).
- At any moment, every family has exactly one **on-duty admin**, derived by a single function `on_duty(family, now)` from three config tables (shifts, coverage rules, absences). Nothing else decides who sees a message.
- Coverage admins (مریم، رضوی) handle families outside the owner's shift. مریم covers **all** owners on Friday. Absent admin → configured backup. Nobody on duty → "Unattended" list for the manager (رقیہ).
- **No shared queue. No automatic routing/distribution. No AI deciding priority.**
- One continuous **thread** per family (WhatsApp-like). Topics are **cases** layered on the thread.
- Priority is not a stored field. It is a computed **attention score** from live signals, shown as a readable reason.
- Workload is a weighted sum of real work units. **Number of families is never part of workload.**
- All thresholds/weights/windows live in a `config` table. Every number below is an initial hypothesis.

2. Roles

Role	Who	Can
admin (owner)	دینا، زینب، رحاب، أسماء	Reply, manage cases/tasks/follow-ups for families where <code>on_duty() == me</code> ; internal notes anywhere; "reply as assist" when allowed
coverage	مریم، رضوی	Same permissions as admin. Role label only. Sees covered families in a separate inbox section
manager	رقیہ	Everything + transfer ownership, edit shifts/coverage/absences, edit config weights, read audit log, offboard admin

Role	Who	Can
finance / technical / academic	internal staff	See and complete tasks assigned to them only. Never message families
system	automation	Deterministic templates and timers only

Family-side contacts have capability flags: `can_message`, `can_view_progress`, `can_manage_schedule`, `can_manage_billing`, `can_manage_contacts`, `can_cancel`, with 4 presets (Primary Guardian / Billing Authorized / Authorized Contact / Secondary Read-only). Store flags, display presets. At least one Primary Guardian per family.

3. Data model

Core

```
family      id, display_name, owner_id FK staff NOT NULL, primary_contact_id,
tier(standard|priority), tier_reason,
            state(onboarding|active|at_risk|renewal_due|paused|churned), state_reason,
state_changed_at,
            language, manual_flag(urgent|null), manual_flag_reason, created_at
contact     id, family_id, app_user_id, name, relationship, role_preset, can_* flags (6),
is_active
learner     id, family_id, name, level, teacher_id, schedule_ref, next_class_at,
last_attended_at, consecutive_absences
subscription id, family_id, learner_id NULL, plan, status, ends_at, renewal_due_at,
last_payment_status, last_payment_at
family_note id, family_id, author_id, body, pinned, created_at
```

Conversation

```
thread      id, family_id UNIQUE, last_customer_message_at, last_staff_message_at,
last_staff_id,
            needs_reply (derived: last_customer > last_staff), sticky_handler_id NULL,
sticky_until NULL
message      id, thread_id, case_id NULL, author_type(contact|staff|system), author_id,
            on_behalf_mode(owner|coverage|assist|escalation), body, attachments jsonb,
            visibility(customer|internal), created_at          -- immutable
case         id, thread_id, family_id, learner_id NULL,

type(technical|billing|schedule|renewal|cancellation|complaint|onboarding|at_risk|academic|general),
            severity(low|medium|high) NULL, is_blocking bool,
            status(open|waiting_customer|waiting_internal|scheduled|resolved|closed),
            handler_id NULL, owner_locked bool, due_at NULL, follow_up_reason,
            escalation_level int, escalated_to_id NULL, opened_by, resolved_at, closed_at,
            reopen_count, root_cause, resolution_summary
task         id, case_id, family_id, type(finance|technical|academic|other), title, details,
owner_id,
            status(open|in_progress|done|cancelled), due_at, result, created_by, created_at,
done_at
handoff      id, thread_id, from_staff_id, to_staff_id,
reason(shift_end|absence|friday|assist|escalation|sticky_expired),
            summary, note, created_at, acknowledged_at
```

Staff & coverage config

```
staff        id, name, role(admin|coverage|manager|finance|technical|academic),
presence(online|away|offline),
            last_activity_at, capacity_override NULL, is_active, left_at
shift        id, staff_id, days[], starts, ends, valid_from, valid_to
coverage_rule id, covering_id, covered_id NULL (NULL = ALL owners), days[],
            window(outside_owner_shift|all_day|custom), custom_from, custom_to, priority int,
valid_from, valid_to
absence      id, staff_id, from, to, backup_id NULL, type(planned|sick|auto_detected),
activated_by
```

Logs & config

```
event_log    id, family_id, case_id NULL, actor_type, actor_id, type, payload jsonb, at          --
append-only
audit_log    id, actor_id, action, entity, entity_id, before jsonb, after jsonb, reason NOT
NULL, at    -- append-only, manager-only read
config       key, value, scope, updated_by, updated_at
family_state_cache family_id, on_duty_id, attention, bucket, top_reason, computed_at    --
recomputed on every family event + every minute for now/today buckets
```

Invariants

- `family.owner_id` changes only through `transfer_ownership(family, to, reason)` → writes `audit_log` + notifies family.
- A family appears in exactly one inbox at any moment (or in the manager's Unattended list). Never two, never zero.

- Every staff message carries `on_behalf_mode`.
- `event_log` and `audit_log` exist from day one. Audit write happens in the same transaction as the sensitive action.
- Relationship case types (`renewal`, `cancellation`, `complaint[severity=high]`, `onboarding`, `at_risk`) get `owner_locked=true` automatically (list in config).

4. Coverage engine

```
on_duty(family, now):
    owner = family.owner
    chain = [owner] + coverage_rules(owner, now).sorted_by(priority).map(r => r.covering)
    for admin in chain:
        if is_absent(admin, now):
            backup = absence(admin, now).backup
            if backup and not is_absent(backup, now) and in_shift(backup, now): return backup
            continue
        if in_shift(admin, now): return admin
    return NONE # → Unattended list + manager alert. Never silently assigned.
```

Stickiness: if a staff member replied to the thread within `config.handoff.grace_minutes` and is still online, the thread stays with them past shift end (`thread.sticky_handler_id/sticky_until`). On expiry → new on-duty admin + handoff record.

Shift-end flow: 1. `N` minutes before shift end: banner to owner ("3 families waiting, 2 follow-ups → snooze to my next shift?"). 2. At shift end: every family with `needs_reply=true` appears in coverage inbox with an auto handoff card (last 3 messages, open cases, owner note, subscription, next class). Quiet families do not move. 3. Coverage replies are tagged `on_behalf_mode=coverage`. 4. Next morning: owner sees "What happened while you were away" (handoffs + summaries).

Absence auto-detect: admin in shift, no activity for `config.absence.auto_detect_minutes`, families waiting → warn manager with one-click "activate backup". **Do not auto-activate in MVP.**

Initial config (illustrative, editable in settings UI): - Owners: Sat–Thu 09:00–17:00. `مريم`: Sat–Thu 17:00–23:00 + Fri 10:00–22:00. `رضوى`: Sat–Thu 17:00–23:00. - Rules: `مريم`→ALL on Fri (priority 1); 2→`مريم` owners & 2→`رضوى` owners outside owner shift (priority 1; which owners is a setting); `رقية`→ALL any time (priority 99, surfaces as Unattended).

5. Handling rules (who may act)

Situation	Rule
Owner in shift	Everything is hers
Owner off-shift, family needs reply	On-duty coverage handles, unless case is <code>owner_locked</code> and not urgent → coverage acknowledges only and it's deferred to owner's next shift

Situation	Rule
<code>owner_locked</code> but urgent (blocking, class now, payment failed, customer wants to renew <i>now</i>)	Coverage acts on the transactional part (send renewal link, record request), never closes the case, flags owner
Any admin	May open any family and write internal notes any time
"Reply as assist"	Allowed only if family is in NOW bucket, waited > 50% of response target, and on-duty admin hasn't opened it — or on-duty admin explicitly requested help. Tagged <code>assist</code> , on-duty notified
Escalation to manager (auto)	Unattended family; response target fully breached with no open; complaint severity=high; same case type 3rd time in 30d; admin at HIGH workload receives a new NOW item; Friday coverage over workload threshold
Escalation (manual)	Button with reason; customer asks for a manager
Nobody on shift (after 23:00)	Auto-reply with honest next-shift time; family shown in manager's "Overnight" section; response clock starts at next covering shift

Coverage **never**: changes owner, closes relationship cases, promises discounts/exceptions.

6. Attention model (replaces P1/P2/P3)

$\text{attention}(\text{family}) = \sum \text{signal_points} \times \text{tier_multiplier}$. Signals are computed from data — no text/AI reading in MVP.

Signal	Initial points
Customer message unanswered	+30, +1/min waiting up to +40
Class in progress or within 30 min AND message	+40
Blocking issue (customer chose "can't join" / admin set <code>is_blocking</code>)	+35
Payment failed within 48h	+20
Renewal due within window	+10 → +25 at 3 days (owner only)
Complaint case open	+25 (+15 if severity high)
Family state <code>at_risk</code>	+15
Same case type within 30 days	+15
Response target: 70% elapsed / breached	+20 / +40
Follow-up due now	+10
Manual flag urgent / escalated	+50
<code>tier=priority</code>	×1.2 (multiplier, not addition)

Buckets: **NOW** (≥60) · **TODAY** (≥25) · **WAITING ON FAMILY** (last message from staff, nothing due) · **QUIET**. Show `top_reason` as text ("class started 5 min ago"), never a number/label.

Response targets (business-hours, initial): NOW-blocking 10/15/20 min (day/night/Friday); NOW-other 30/45/60 min; TODAY before shift end (or deferred to owner). Add an inbox button "this order is wrong" that logs what the admin would have done — calibration data.

7. Workload engine

```
workload(admin, now) =  $\sum \text{weight}[\text{unit}] \times \text{count}(\text{unit where on\_duty}(\text{family}, \text{now}) == \text{admin})$ 
```

Units (initial weights): needs_reply 1.0 · active case 1.0 · follow-up due today 0.7 · SLA risk 1.5 · waiting_internal 0.3 · renewal in window 0.5 · complaint open 1.5 · technical open 1.2 · at_risk family 0.8. Covered families count fully on the coverage admin during her shift.

Levels: LOW < 8 · MEDIUM < 15 · HIGH ≥ 15, plus HIGH if `needs_reply_count ≥ config.reply_burst`. All in config.

Log per case from day one: staff message count, time to first reply, handling time excluding waiting_customer, reopen count → used after 60–90 days to derive weights from real minutes.

8. Screens

Admin inbox (list of families, not tickets)

Sections: **NOW** (attention desc, then oldest) · **TODAY** (by due) · **WAITING ON FAMILY** (collapsed) · **QUIET** (search only) · **Families I'm covering now** (coverage/backup) · **What happened while you were away** (owner, morning). Each row: family name, top_reason, wait time, colored dot.

Family screen

- Center: single continuous thread (all contacts, all staff, system cards for payment failed / class started / handoff / task done), internal notes inline in a different color, open **case cards** on top (click to filter). Composer: reply, canned replies, attach, "reply + snooze", "reply + resolve". Actions: new case, follow-up date, internal task, escalate, "I'll keep this" (pin handler), "for owner" (defer), "reply as assist" (conditional).
- Side panel (always visible): owner + on-duty if different; contacts with roles; learners (level, teacher, next class, last attended, consecutive absences); subscription (plan, ends, last payment, renewal in N days); family state + reason; pinned notes; last 5 cases; open tasks.

Manager dashboard (رقيه)

- Header: Unattended count (target 0), open escalations.
- Team now: per admin → presence, NOW count, TODAY count, late replies, workload score, level, inactivity warning.
- Coverage: tonight (who → whom), next Friday, gaps in next 7 days (absence without backup, missing rule).

- Needs your action: escalations, auto-detected absence (with "activate backup"), HIGH admin receiving NOW item.
- This week: renewals (due/renewed/no reply/refused), at_risk (by reason), response-target compliance, median first reply by shift.
- Actions: transfer ownership, activate backup, edit shifts/coverage/absences, edit weights/thresholds, offboard admin (one click: deactivate, transfer families evenly or to a named admin, flag orphaned coverage rules, notify families, audit).

Tasks screen (finance/technical/academic)

Only their tasks, with case context. Completing the last open task on a case → case `waiting_internal` → `open`, family moves to TODAY with reason "inform family of finance result". Task SLA: 1 business day; delays show on manager dashboard by department.

Customer (help screen in app)

Owner name + photo ("Your Jawwid team: دينا"). Their open topics in plain status. "Continue this topic" / "New topic" (pick from ~5 categories, optional "I can't join the class"). Self-service buttons by permission: my subscription, next class, invoice, renewal link. Honest expected reply time from current/next shift. No internal labels.

9. Automation (deterministic only)

- Instant acknowledgement with honest reply-time from `on_duty()` /next shift; out-of-hours reply.
- Batch wait ~60s before acknowledging multiple rapid messages.
- Self-service: subscription status, next class, invoice resend, renewal link (billing-permitted contacts only).
- Lifecycle triggers (open a case for the owner): onboarding on first payment; renewal at `renewal_due_days_before` (14); at_risk on payment failure or `inactivity_days` (14) without attendance. Cancellation intent → at_risk + case for owner.
- Timers: no-reply reminders [24h, 72h]; operational auto-resolve after 168h (never for owner_locked types → becomes owner follow-up); resolved → closed after `reopen.window_hours` (72); customer message inside window reopens automatically (same handler if on duty).

10. MVP scope (build exactly this)

1. family / contact (flags) / learner (next_class_at) / subscription / family_note; import current owner assignments as-is.
2. thread + message + case + task + handoff with the invariants above.
3. Coverage engine: shift / coverage_rule / absence tables + settings UI + `on_duty()` + stickiness + handoff cards + "while you were away" + Unattended list + absence auto-detect (suggest only).
4. Admin inbox and family screen as specified; end-of-shift banner.
5. Attention model + workload engine with all points/weights/thresholds in `config`; `family_state_cache`; "this order is wrong" button; calibration logging.

6. Tasks screen for departments.
7. Deterministic automation above.
8. Manager dashboard + actions + one-click offboarding.
9. event_log, audit_log, weekly report (first reply by shift, target compliance, renewals, at_risk, workload per admin).

Not in MVP: shared queue / auto-distribution · any AI (no summaries, no drafting, no classification) · skills engine · weighted routing · predictive churn/risk · WFM · auto-activating backups · CSAT · QA rubrics · multiple specialized queues · multi-team/pod structures.

11. Later phases (design for, don't build)

Phase 2 (after 60–90 days of data): recalibrate weights/thresholds from logs; internal AI copilot (handoff summary first, then family summary, suggested reply, case-type suggestion — read-only, outputs stored as internal); text-derived attention signals (can't join / cancel / anger) as *signals*, not decisions; richer at_risk rules; CSAT; broader self-service; optional auto-activate backup; night-balance suggestions to manager; targeted assist suggestions.

Phase 3 (scale): teams (staff.team_id , coverage_rule.covered_team_id); coverage pools (least-loaded *within* a small pool — the only place "least loaded" ever appears); specialist desks via tasks; optional family.retention_owner_id ; AI with explicit allowlisted actions via an ai_authorization table; light WFM from event_log volume forecasts. None of this should require changing family/thread/case/coverage entities.

12. Non-negotiables checklist for code review

- [] No code path assigns a message/family to a staff member other than via on_duty() (+ stickiness, assist, escalation — all logged with reason).
- [] No queue table, no round-robin, no "least loaded" outside Phase 3 pools.
- [] Owner changes only via transfer_ownership(); writes audit_log in the same transaction.
- [] thread is UNIQUE per family; cases never create a second thread.
- [] Every staff message has on_behalf_mode.
- [] Attention and workload are computed, never user-entered; all constants read from config.
- [] Family can never be in two inboxes or in none (except Unattended).
- [] Coverage cannot close owner_locked cases or change owners.
- [] No customer-facing AI output. No AI-initiated state changes.
- [] Every number in this brief is treated as a config default with a comment "initial hypothesis".